

DocCompare - Client Handoff Report

Document Knowledge Base, RAG Chat, and Acceptance Criteria Gap Analysis

Prepared for Client Handoff

April 27, 2026

Contents

Executive Summary	3
Application Purpose	3
Client Use Cases and Expansion Opportunities	3
Software Requirements and User Story Gap Analysis	3
Contract Review and Risk Analysis	4
Policy and SOP Change Impact Analysis	4
Regulatory and Compliance Audit Support	4
Vendor and RFP Evaluation	5
High-Level Capabilities	5
Knowledge Base Creation	5
New Document Comparison	5
Acceptance Criteria-Level Gap Analysis	5
Knowledge Base Chat	5
Role-Based Access Control	6
Architecture Overview	6
System Components	6
System Architecture Diagram	6
Architecture Flow	6
Detailed Workflow	8
Knowledge Base Ingestion Workflow	8
Chunking Model	8
Document Comparison Workflow	8
Chat Workflow	9
Frontend User Guide	9
Login	9
Admin: Add Documents to the Knowledge Base	9
Admin: Delete a Knowledge Base Document	10
User: Compare a New Document	10
User: Chat with the Knowledge Base	10
Settings: Select GPT or Ollama	12
Configuration Guide	14
Main Configuration File	14
Important Environment Variables	14
Docker Configuration Notes	15
Runtime LLM Settings	15

Docker Startup Guide	15
Prerequisites	15
Start the Application with Docker	16
Access the Application	16
Stop the Application	16
View Logs	16
Reset All Docker Data	16
Enter Backend Container	16
Enter Database Shell	16
Local Development Guide	17
Backend Local Run	17
Frontend Local Run	17
API Summary	17
Data Model	18
document_chunks	18
users	18
sessions	18
Security and Access Notes	18
Operational Notes	19
Ingestion Reliability	19
Embedding Consistency	19
Knowledge Base Metadata	19
Data Persistence	19
Testing and Verification	19
Basic Docker Health Check	19
Login Test	19
Knowledge Base List Test	20
Settings Ping Test	20
Client Handoff Checklist	20
Future Enhancements and Product Roadmap Ideas	20
Platform Enhancements	20
Domain Expansion Ideas	21
Conclusion	21

Executive Summary

DocCompare, also represented in the codebase as Doc Gap Analysis, is a document intelligence application that converts business, specification, and requirements documents into a searchable knowledge base, then compares new specification documents against that knowledge base to identify acceptance criteria-level coverage, gaps, partial matches, useful additions, and conflicts.

The application uses a Retrieval-Augmented Generation (RAG) architecture. Existing documents are ingested, parsed, structured with an LLM, chunked into semantic units, embedded, and stored in PostgreSQL with pgvector. When a new document is uploaded for comparison, the system extracts its user stories and acceptance criteria, retrieves related content from the knowledge base, and asks an LLM to produce a structured gap analysis report.

The application also includes a knowledge base chat experience. Users can ask questions about stored documents, and the system retrieves the most relevant chunks before generating a source-backed answer. This gives users a practical way to verify what is already present in the knowledge base before or after running a document comparison.

The system supports two user roles:

- Admin: can upload, delete, and manage files in the knowledge base.
- User: can run document comparisons and use chat against the knowledge base.

The LLM provider is OpenAI-compatible. The production default is OpenAI/GPT. The UI also includes an Ollama-compatible configuration path for future local-model experimentation. Ollama is intentionally marked experimental because some local models may not consistently return the strict JSON required by ingestion and gap analysis workflows.

Application Purpose

The application is designed for teams that maintain a large set of existing requirements, specifications, user stories, BRDs, or implementation documents and need to evaluate how a new document compares against that existing baseline.

Typical client use cases include:

- Determining whether a new specification is already covered by existing project documentation.
- Identifying missing acceptance criteria before implementation begins.
- Finding partial matches where existing requirements cover the same area but differ in detail.
- Highlighting good-to-have additions that improve the current scope but are not critical blockers.
- Detecting conflicts between new requirements and existing documented behavior.
- Asking natural-language questions about the knowledge base and receiving answers with source references.

The key value of the application is that it does not simply compare documents at a raw text level. It extracts and compares structured requirements, especially acceptance criteria, which is much more useful for business analysts, product owners, QA teams, solution architects, and delivery leads.

Client Use Cases and Expansion Opportunities

The platform supports a primary software requirements comparison workflow and can also be extended into other document-heavy business domains. This is possible because the architecture is prompt-driven and schema-driven: with the right extraction prompt, metadata model, and verdict definitions, the same RAG comparison engine can be adapted to different document categories.

Software Requirements and User Story Gap Analysis

This is the current primary use case. Existing BRDs, user stories, and acceptance criteria are uploaded into the knowledge base. A new specification is compared against that baseline to identify covered, partial, missing, good-to-have, or conflicting requirements.

This workflow is most relevant for:

- Business analysts.

- Product owners.
- QA teams.
- Delivery managers.
- Solution architects.

Contract Review and Risk Analysis

A legal or procurement team can upload standard contract templates, approved clauses, or existing master service agreements into the knowledge base. Incoming vendor or client contracts can then be compared against that baseline.

Potential output categories:

- Missing clauses.
- Conflicting terms.
- Deviations from standard language.
- High-risk obligations.
- Non-standard payment, termination, liability, or data protection terms.

Implementation adaptation:

- Update the extraction prompt to target contract clauses.
- Add metadata fields such as `clause_type`, `risk_level`, `obligation_owner`, and `jurisdiction`.
- Redefine verdicts around contractual risk and compliance.

Policy and SOP Change Impact Analysis

Operations teams can upload current SOPs, internal policies, process documents, and work instructions into the knowledge base. New corporate policies can then be checked against current procedures to identify operational gaps and conflicts.

Potential output categories:

- SOPs that require updates.
- Policy conflicts.
- Missing procedural controls.
- Department-specific impacts.
- Required retraining or communication items.

Implementation adaptation:

- Update the extraction prompt to parse policy directives and procedural steps.
- Add metadata such as `department`, `policy_reference`, `control_owner`, and `effective_date`.
- Weight gaps by operational impact.

Regulatory and Compliance Audit Support

A healthcare, financial services, or regulated operations team can upload compliance standards and internal controls into the knowledge base. New BRDs, SOPs, or system requirements can then be checked for regulatory gaps before audit or implementation.

Potential output categories:

- Missing regulatory controls.
- Partial compliance.
- Conflicts with required standards.
- Audit-readiness gaps.
- Criticality-ranked findings.

Implementation adaptation:

- Update the extraction prompt to target regulatory clauses and control obligations.
- Add metadata such as `regulation`, `control_id`, `severity`, and `evidence_required`.
- Adjust the gap report to prioritize high-severity compliance gaps.

Vendor and RFP Evaluation

Procurement teams can upload internal requirements, RFP criteria, or evaluation rubrics into the knowledge base. Vendor proposals can then be compared against that standard to produce a structured fit/gap report.

Potential output categories:

- Met requirements.
- Partially met requirements.
- Missing capabilities.
- Vendor claims requiring clarification.
- Comparative compliance scores.

Implementation adaptation:

- Update the extraction prompt to identify vendor capability claims.
- Add metadata such as `vendor_id`, `capability_area`, `requirement_priority`, and `evidence_status`.
- Extend the output with a 0-100 compliance score for proposal comparison.

High-Level Capabilities

Knowledge Base Creation

Administrators upload existing documents into the knowledge base. Each document is processed through a pipeline:

1. The document is parsed into text or markdown.
2. An LLM extracts structured user stories and acceptance criteria.
3. The extracted information is split into story chunks and acceptance criteria chunks.
4. Embeddings are generated for each chunk.
5. The chunks and embeddings are stored in PostgreSQL using pgvector.

This creates a reusable semantic knowledge base for comparison and chat.

New Document Comparison

Users upload a new document, such as a new specification or BRD. The application:

1. Parses the new document.
2. Extracts structured stories and acceptance criteria.
3. Searches the knowledge base for related stories.
4. Pulls the matching acceptance criteria from the knowledge base.
5. Runs an LLM-based gap analysis.
6. Displays a clear acceptance criteria-based report in the frontend.

Acceptance Criteria-Level Gap Analysis

The gap analysis produces structured verdicts:

- Covered: existing knowledge base already satisfies the new criterion.
- Partial: existing knowledge base addresses the same area but misses one or more details.
- Gap: the new criterion is not meaningfully covered and should be addressed.
- Good to have: the new criterion adds value but is not a blocker.
- Conflict: the new criterion contradicts or is incompatible with existing criteria.

The frontend summarizes these verdicts with counts, coverage percentage, key gaps, recommended additions, and an overall recommendation.

Knowledge Base Chat

The chat feature lets users ask natural language questions about the knowledge base. The system retrieves relevant story or acceptance criteria chunks and sends them to the selected chat LLM. The answer is returned with source references, including document title, story title, acceptance criteria title, similarity score, and content snippet.

Role-Based Access Control

The application supports two roles:

- Admin users manage the knowledge base.
- Regular users run comparisons and chat with the knowledge base.

This keeps source-of-truth knowledge base changes controlled while still allowing broader users to perform analysis.

Architecture Overview

System Components

The application is organized into three primary runtime services:

Service	Technology	Purpose
Frontend	React, Vite, Nginx	User interface for login, KB management, comparison, chat, and settings
Backend API	Python, FastAPI	Application API, ingestion orchestration, comparison, chat, settings, authentication
Database	PostgreSQL 16 with pgvector	Stores users, sessions, document chunks, metadata, and embeddings

The codebase is organized around the same separation:

Area	Path	Description
Backend routes	<code>src/rag_api/routes/</code>	FastAPI route handlers
Backend dependencies	<code>src/rag_api/dependencies.py</code>	Shared dependency wiring for settings, vector store, LLMs, auth
Ingestion pipeline	<code>src/rag_ingest/pipeline.py</code>	Orchestrates parse, extract, chunk, embed, store
Document parser	<code>src/rag_ingest/ingest.py</code>	Uses Docling to convert supported documents to markdown/text
LLM extractor	<code>src/rag_ingest/extractor.py</code>	Sends parsed document text to LLM and validates structured JSON
Chunking	<code>src/rag_ingest/chunking.py</code>	Creates story chunks and acceptance criteria chunks
Vector store	<code>src/rag_ingest/store.py</code>	PostgreSQL/pgvector storage and retrieval layer
LLM provider	<code>src/rag_ingest/llm/openai_provider.py</code>	OpenAI-compatible chat and embedding calls
Frontend app	<code>frontend/src/</code>	React UI components and API client

System Architecture Diagram

The diagram below summarizes the end-to-end system architecture, including the React frontend, role-based access, document parsing, chunking, embedding, PostgreSQL storage, retrieval, LLM analysis, and gap report output.

Architecture Flow

The architecture follows this flow:

React/Vite UI -> Admin/User role -> PDF/DOCX upload -> Document parser -> Chunking -> Embedding -> PostgreSQL/pgvector -> Retrieval -> LLM analysis -> Gap report

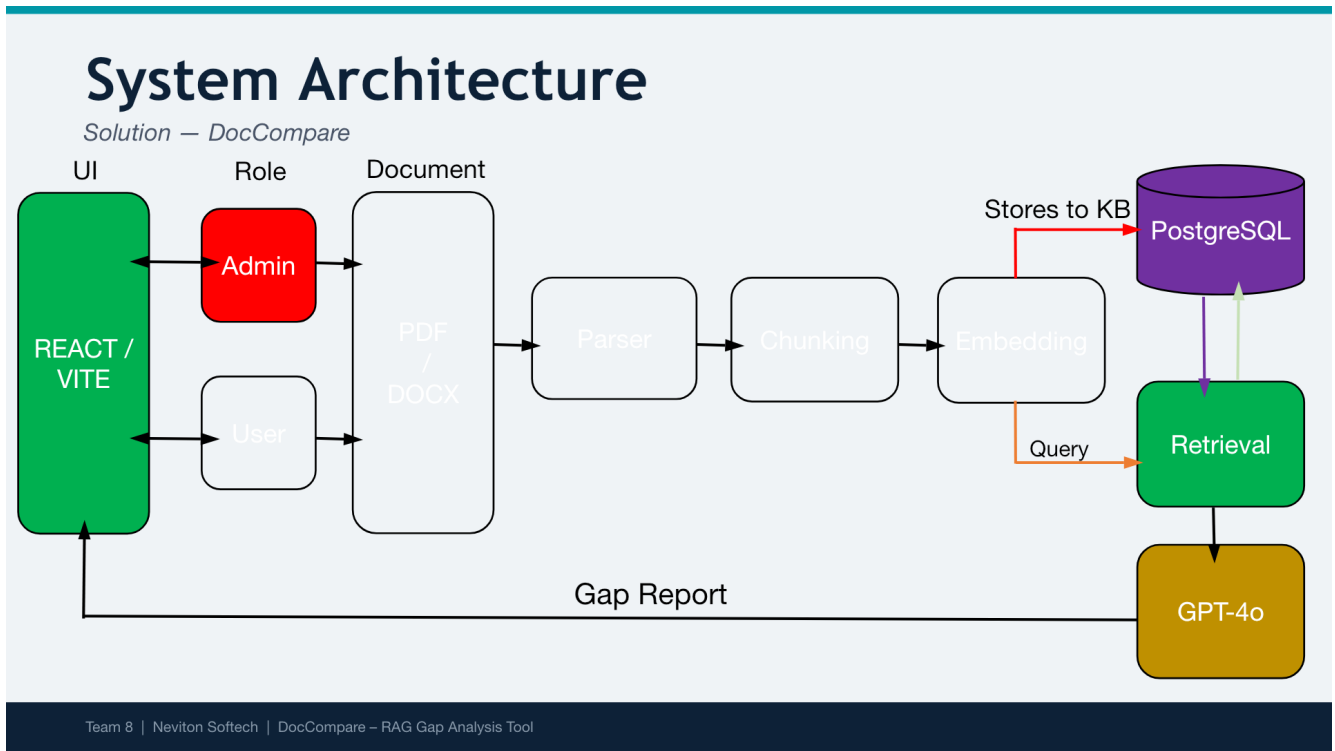


Figure 1: DocCompare system architecture

This is consistent with the application implementation, with the following implementation-specific details:

Architecture Area	Implementation Detail
React/Vite	Frontend is implemented in <code>frontend/src</code> and served by Nginx in Docker
Admin/User role	Backend session authentication supports <code>admin</code> and <code>user</code> roles
PDF/DOCX upload	Knowledge base and comparison upload flows support business document ingestion
Document Parser	Docling converts supported files into text/markdown
Chunking	The backend creates story chunks and acceptance criteria chunks from LLM-extracted JSON
Embedding	OpenAI text embeddings are generated for stored chunks
PostgreSQL	PostgreSQL with pgvector stores chunks, embeddings, users, and sessions
Retrieval	Vector similarity search retrieves related story and criteria chunks
GPT-4o	GPT/OpenAI-compatible LLM calls are used for extraction, comparison, and chat
Gap Report	The frontend renders acceptance criteria-level verdicts and summary metrics

GPT or another configured OpenAI-compatible model performs semantic extraction and gap-analysis reasoning. The embedding step uses the configured embedding model, currently `text-embedding-3-small` by default. The chunking step is deterministic application logic over the LLM-extracted JSON, not a separate model training process.

Detailed Workflow

Knowledge Base Ingestion Workflow

Knowledge base ingestion is initiated from the frontend Knowledge Base panel by an admin user. The backend endpoint is:

POST /api/knowledge-base/upload

The workflow is:

1. Admin selects one or more PDF or DOCX files.
2. The frontend sends the files to the backend as multipart form data.
3. The backend assigns a unique file ID and stores the uploaded file under the configured upload directory.
4. Metadata is written to `data/kb_metadata.json`.
5. A background task starts the ingestion pipeline.
6. The pipeline parses the document using Docling.
7. The LLM extracts structured document JSON.
8. The chunker creates story chunks and acceptance criteria chunks.
9. The embedding provider creates vector embeddings.
10. The vector store writes chunks and embeddings to PostgreSQL/pgvector.
11. The file status changes to **ready**.

Processing status values shown in the UI include:

- `processing`
- `docling`
- `llm`
- `chunking`
- `embedding`
- `ready`
- `error`

This status tracking helps users understand where the file is in the ingestion lifecycle.

Chunking Model

The system intentionally separates content into two chunk types:

Chunk Type	Purpose
Story chunk	Used for semantic matching between new stories and existing knowledge base stories
Criteria chunk	Used for detailed comparison of acceptance criteria under matched stories

This design improves retrieval quality. The application first finds semantically similar stories, then retrieves the associated acceptance criteria for detailed gap analysis.

Document Comparison Workflow

The comparison workflow uses two API calls:

1. POST /api/documents/upload
2. POST /api/documents/compare

The upload step parses the new document and searches the knowledge base for related content. The compare step sends the extracted acceptance criteria and retrieved knowledge base context to the comparison LLM.

The frontend displays the final structured result using the Gap Analysis Dashboard. The report includes:

- New document title

- Existing matched document title
- Total acceptance criteria count
- Covered count
- Partial count
- Gap count
- Good-to-have count
- Conflict count
- Coverage percentage
- Key gaps
- Key additions
- Recommendation
- Per-acceptance-criteria comparison records

Chat Workflow

The chat endpoint is:

POST /api/chat

The workflow is:

1. User enters a question in the chat panel.
2. The backend embeds the question.
3. The vector store searches across story and criteria chunks.
4. The top matching chunks are formatted as context.
5. The chat LLM produces a natural-language answer.
6. The frontend displays the answer and expandable sources.

Source visibility is important for client confidence. Users can inspect which documents and chunks supported the answer.

Frontend User Guide

Login

1. Open the application in the browser.
2. Enter a username and password.
3. On success, the application opens the main workspace.

Default development credentials seeded by the app are:

Username	Password	Role
admin1	admin1pass	admin
admin2	admin2pass	admin
trial1	trial1pass	admin
user1	user1pass	user
user2	user2pass	user

Important: these credentials are for local development/demo use only and should be changed before production use.

Admin: Add Documents to the Knowledge Base

1. Log in as an admin.
2. Open the Knowledge Base panel on the left.
3. Click **Add files**.
4. Select supported documents, typically PDF or DOCX.
5. Wait for the status to move through parsing, extraction, chunking, and embedding.

6. When the status becomes **Indexed**, the document is ready for comparison and chat.

Only admin users see the upload control.

Admin: Delete a Knowledge Base Document

1. Log in as an admin.
2. Hover over a file in the Knowledge Base panel.
3. Click the delete icon.
4. The backend removes the file metadata and deletes stored chunks for that source from pgvector.

Only admin users see the delete control.

User: Compare a New Document

1. Log in as a user or admin.
2. Click or drag a document into the comparison upload area.
3. Wait for processing.
4. The app extracts the document, retrieves matching knowledge base chunks, and generates the gap analysis.
5. Review the dashboard.

The dashboard should be read as follows:

Dashboard Area	Meaning
Total ACs	Total acceptance criteria identified in the new document
Covered	New criteria already satisfied by existing KB criteria
Partial	Related coverage exists, but details differ or are incomplete
Gaps	Missing requirements that likely need action
Good to have	Useful additions that may not be critical
Coverage percentage	Covered plus partial items divided by total new criteria
Key gaps	Most important missing items
Recommended additions	Valuable enhancements suggested by the comparison
Overall recommendation	Narrative summary for delivery or analysis teams

Users can filter by verdict to focus on gaps, partials, or covered items.

The detailed view expands each user story and shows the acceptance criteria-level reasoning behind each verdict, including the matched knowledge base criterion and the model's explanation.

User: Chat with the Knowledge Base

1. Click **Ask KB** in the header.
2. Type a question about uploaded documents, requirements, user stories, or acceptance criteria.
3. Submit the question.
4. Review the answer.
5. Expand **Sources** to see the supporting chunks.

Example questions:

- “Which documents mention vendor qualification?”
- “What acceptance criteria exist for reminders?”
- “Do we already have SAP synchronization requirements?”
- “Summarize the authentication requirements in the knowledge base.”
- “Which criteria mention audit logging?”

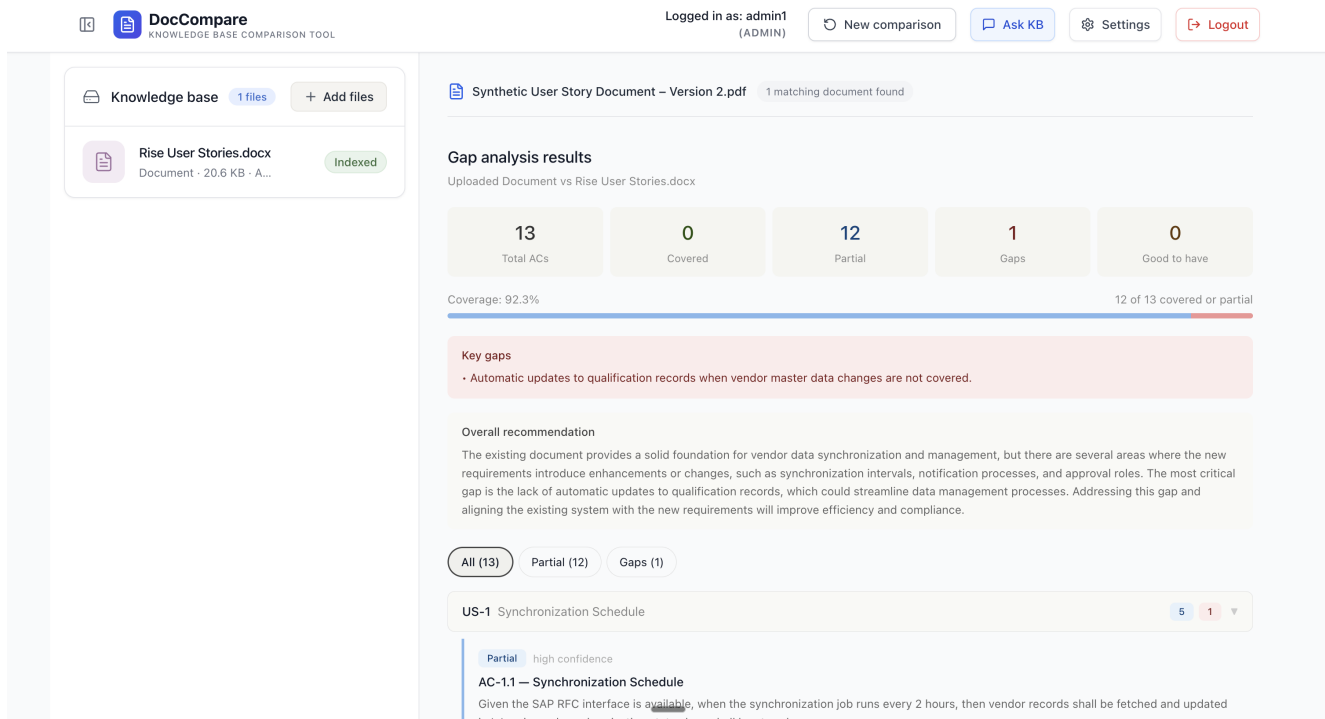


Figure 2: Gap analysis dashboard showing coverage, key gaps, and recommendation

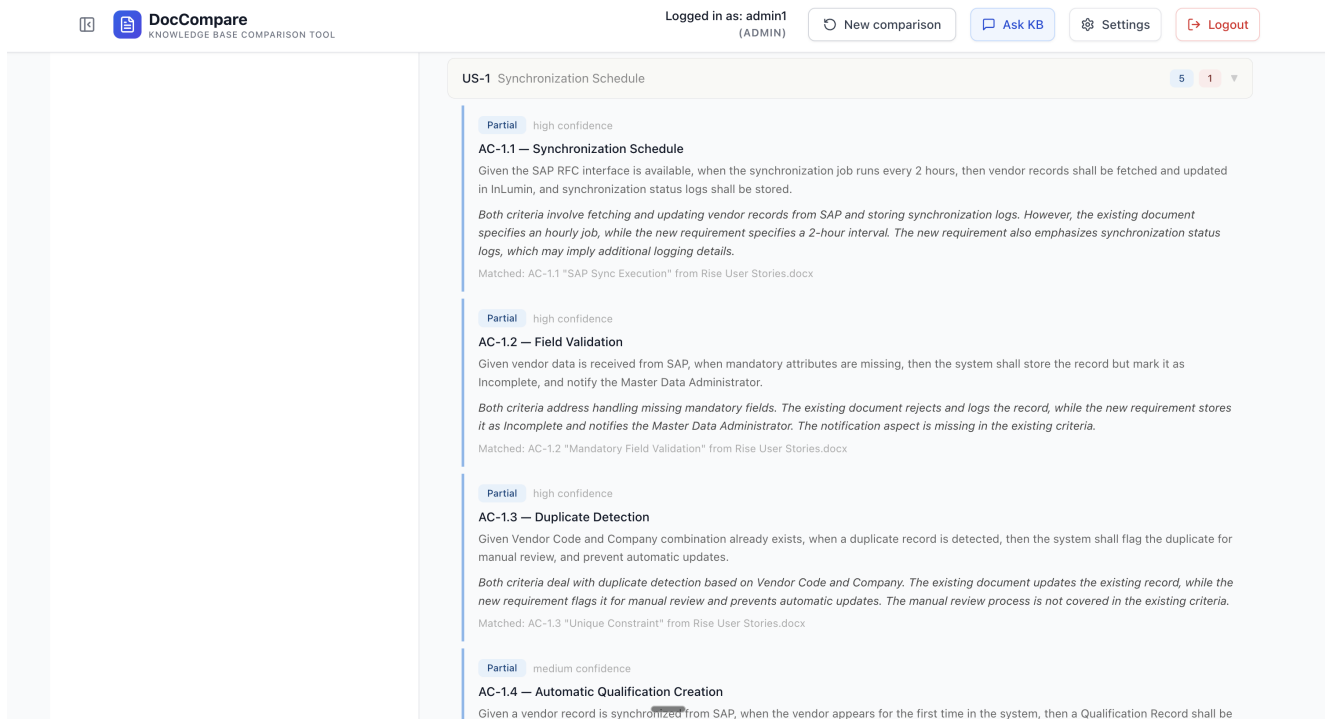


Figure 3: Acceptance criteria-level gap analysis detail

Settings: Select GPT or Ollama

Open the Settings modal from the application header.

The settings UI supports:

- Default model.
- Default base URL.
- Ingestion model and base URL.
- Document comparison model and base URL.
- Chat model and base URL.
- Connection test for each use case.

The screenshot shows a modal titled "LLM settings" with a close button (X) in the top right corner. Inside the modal, there is a light blue informational box at the top stating: "Production default: OpenAI (official API). Per-feature values that match the defaults below are stored as inherited. Ollama is **experimental** and may return lower-quality or incompatible JSON for some flows."

Below this is a section titled "Default (all features)" with a subtitle "Used when a feature's model or base URL matches these values, and for CLI / env fallbacks." It contains two input fields: "Default model" with the value "gpt-4o" and "Default base URL" with the value "Empty = OpenAI default endpoint". Below these fields are two buttons: "Preset: OpenAI" (highlighted in blue) and "Preset: Ollama (experimental)".

The next section is titled "Ingestion" with a subtitle "Structured extraction when adding knowledge base documents." and a "Test connection" button with a checkmark icon. It features two large buttons: "GPT (OpenAI)" (highlighted in blue) and "Ollama (experimental)". Below these are three input fields: "Model" with a dropdown menu showing "gpt-4o", "Base URL" with the text "Match default below, or OpenAI / Ollama host", and a note at the bottom stating "Leave identical to **Default base URL** below to inherit the deployment default."

At the bottom right of the modal are two buttons: "Cancel" and "Save settings" (highlighted in blue).

Figure 4: LLM settings for default and ingestion model configuration

Recommended production setup:

- Provider: OpenAI-compatible.
- Default model: `gpt-4o` or an approved production model.
- Base URL: blank for the official OpenAI endpoint.
- Embedding model: `text-embedding-3-small`.

LLM settings

Document comparison

Gap analysis JSON when comparing an upload to the knowledge base.

Endpoint reachable (models.list), model='gpt-4o' (982 ms)

GPT (OpenAI)

Ollama (experimental)

Model

gpt-4o

Base URL

Match default below, or OpenAI / Ollama host

Leave identical to **Default base URL** below to inherit the deployment default.

Chat

RAG answers over retrieved chunks.

GPT (OpenAI)

Ollama (experimental)

Model

gpt-4o

Base URL

Match default below, or OpenAI / Ollama host

Leave identical to **Default base URL** below to inherit the deployment default.

Cancel

Save settings

Figure 5: LLM settings for comparison and chat model configuration

Experimental Ollama setup:

- Base URL: `http://localhost:11434/v1`.
- Model: installed Ollama model name, for example `llama3.1:latest`.

Note: Ollama may not reliably return strict JSON for ingestion and comparison. It is best treated as future-ready experimentation unless validated with the client's own document set.

Configuration Guide

Main Configuration File

The main environment configuration is copied from:

`.env.example`

to:

`.env`

Recommended initial setup:

```
cp .env.example .env
```

Then update the required values:

- `OPENAI_API_KEY`
- `PG_PASSWORD`

Important Environment Variables

Variable	Required	Default	Description
<code>OPENAI_API_KEY</code>	Yes	<code>none</code>	API key used for OpenAI-compatible LLM and embedding calls
<code>PG_HOST</code>	No	<code>localhost</code>	Database host; use <code>db</code> inside Docker backend container
<code>PG_PORT</code>	No	<code>5432</code>	Database port; Docker exposes local Postgres at <code>5433</code>
<code>PG_DATABASE</code>	No	<code>rag_gap</code>	PostgreSQL database name
<code>PG_USER</code>	No	<code>postgres</code>	PostgreSQL username
<code>PG_PASSWORD</code>	Yes	<code>none</code>	PostgreSQL password
<code>PG_POOL_MIN</code>	No	<code>1</code>	Minimum database connection pool size
<code>PG_POOL_MAX</code>	No	<code>10</code>	Maximum database connection pool size
<code>LLM_PROVIDER</code>	No	<code>openai</code>	LLM provider name; current implementation supports OpenAI-compatible provider
<code>LLM_MODEL</code>	No	<code>gpt-4o</code>	Default LLM model
<code>LLM_MAX_TOKENS</code>	No	<code>16384</code>	Maximum output tokens for LLM calls
<code>EMBEDDING_PROVIDER</code>	No	<code>openai</code>	Embedding provider
<code>EMBEDDING_MODEL</code>	No	<code>text-embedding-3-small</code>	Embedding model

Variable	Required	Default	Description
EMBEDDING_DIMENSIONS	No	1536	Vector dimension stored in pgvector
EMBEDDING_BATCH_SIZE	No	100	Embedding batch size
IVFFLAT_LISTS	No	100	pgvector IVFFlat index list count
SEARCH_TOP_K	No	5	Number of top search results
MAX_RETRIES	No	3	Retry attempts for external API calls
RETRY_BACKOFF_MULTIPLIER	No	1.0	Retry backoff multiplier
RETRY_MAX_WAIT	No	30	Maximum retry wait in seconds
DATA_DIR	No	data	Application data directory
UPLOAD_DIR	No	data/uploads/kb	Knowledge base upload directory
METADATA_FILE	No	data/kb_metadata.json	Metadata JSON path

Docker Configuration Notes

When running through Docker Compose:

- The backend connects to PostgreSQL using `PG_HOST=db`.
- PostgreSQL listens inside Docker on 5432.
- PostgreSQL is exposed to the host machine on 5433.
- Backend API is exposed on `http://localhost:8000`.
- Frontend is exposed on `http://localhost:3000`.

Runtime LLM Settings

The application also provides runtime LLM settings through:

`/api/settings/`

These settings are currently in-memory for the running backend process and are not written back to `.env`. They are useful for testing different models or endpoints during a session.

Per-feature model settings are available for:

- Ingestion.
- Comparison.
- Chat.

If a per-feature model or base URL is blank or matches the default, it inherits the default setting.

Docker Startup Guide

Prerequisites

Install:

- Docker.
- Docker Compose.
- Valid OpenAI API key.

Optional for local non-Docker development:

- Python 3.11+.

- Node.js 20+.
- PostgreSQL 16 with pgvector.

Start the Application with Docker

From the repository root:

```
cp .env.example .env
```

Edit `.env` and set:

```
OPENAI_API_KEY=your_openai_api_key  
PG_PASSWORD=your_postgres_password
```

Then start:

```
make up
```

This runs:

```
docker compose up -d --build
```

Access the Application

After startup:

- Frontend: `http://localhost:3000`
- Backend API: `http://localhost:8000`
- PostgreSQL host port: `localhost:5433`

Stop the Application

```
make down
```

This runs:

```
docker compose down
```

View Logs

```
make logs
```

This runs:

```
docker compose logs -f
```

Reset All Docker Data

```
make reset
```

This runs:

```
docker compose down -v
```

Warning: this removes Docker volumes, including the PostgreSQL data volume.

Enter Backend Container

```
make backend-shell
```

Enter Database Shell

```
make db-shell
```


Local Development Guide

Backend Local Run

Create a virtual environment:

```
python3 -m venv .venv
source .venv/bin/activate
```

Install dependencies:

```
pip install -r requirements.txt
```

Copy and configure environment variables:

```
cp .env.example .env
```

For local backend development against the Docker database, set:

```
PG_HOST=localhost
PG_PORT=5433
```

Start backend:

```
python -m uvicorn src.rag_api.app:app --reload --port 8000
```

Frontend Local Run

From the frontend directory:

```
cd frontend
npm install
npm run dev
```

The Vite development server uses port 5173 and proxies `/api` requests to:

`http://127.0.0.1:8000`

API Summary

Method	Endpoint	Purpose	Auth
POST	<code>/api/auth/login</code>	Authenticate and receive session token	No
POST	<code>/api/auth/logout</code>	Revoke session	Yes
GET	<code>/api/auth/me</code>	Return current user profile	Yes
GET	<code>/api/knowledge-base</code>	List KB files	Yes
POST	<code>/api/knowledge-base/upload</code>	Upload and ingest KB files	Admin
DELETE	<code>/api/knowledge-base/{file_id}</code>	Delete KB file and chunks	Admin
POST	<code>/api/documents/upload</code>	Parse new document and retrieve matches	Yes
POST	<code>/api/documents/compare</code>	Generate structured gap analysis	Yes
POST	<code>/api/chat</code>	Ask KB chat question	Yes
GET	<code>/api/settings/</code>	Read LLM settings	Yes
POST	<code>/api/settings/</code>	Update runtime LLM settings	Yes
POST	<code>/api/settings/ping</code>	Test selected LLM endpoint	Yes

Data Model

document_chunks

Stores the RAG knowledge base chunks.

Column	Type	Description
id	serial	Internal primary key
chunk_id	text	Unique chunk identifier
chunk_type	text	story or criteria
content	text	Chunk text
embedding	vector	pgvector embedding
story_id	text	Link between criteria and parent story
metadata	jsonb	Story, AC, and document metadata
source_path	text	Source document identifier/path

users

Stores application users.

Column	Type	Description
id	serial	Internal primary key
user_id	text	Business/application user ID
username	text	Login username
password_hash	text	bcrypt password hash
role	text	admin or user
created_at	timestamp	Created timestamp

sessions

Stores active login sessions.

Column	Type	Description
id	serial	Internal primary key
token	text	Session token
user_id	text	Linked user ID
created_at	timestamp	Session creation timestamp
expires_at	timestamp	Session expiration timestamp

Sessions currently expire after 24 hours.

Security and Access Notes

The application includes practical security foundations:

- Passwords are stored with bcrypt hashing.
- Sessions are server-side records stored in PostgreSQL.
- Protected endpoints require bearer token authentication.
- Admin-only operations are enforced in the backend, not only the UI.
- Knowledge base mutation is restricted to admins.

Production recommendations:

- Replace all default seeded credentials.

- Use HTTPS/TLS in deployed environments.
- Store `.env` secrets securely.
- Restrict CORS origins instead of allowing all origins.
- Rotate OpenAI API keys according to client policy.
- Add audit logging for admin upload/delete operations if required by compliance.
- Consider persistent session/token handling policy based on client security standards.

Operational Notes

Ingestion Reliability

The ingestion flow depends on strict JSON returned by the selected ingestion LLM. OpenAI models are expected to be more reliable for this workflow. Ollama/local models can be tested but may produce invalid JSON unless the selected model is strong at instruction following.

Embedding Consistency

The embedding dimension is configured by `EMBEDDING_DIMENSIONS` and reflected in the pgvector table definition. If the embedding model or dimensions are changed after data exists, the database may need to be rebuilt or migrated.

Knowledge Base Metadata

File-level metadata is stored in:

`data/kb_metadata.json`

The actual searchable chunks are stored in PostgreSQL.

Data Persistence

Docker Compose persists PostgreSQL data in the named Docker volume:

`pgdata`

Uploaded files and metadata are mounted through:

`./data:/app/data`

This means uploaded document files and metadata remain available across container restarts unless removed.

Testing and Verification

Basic Docker Health Check

`docker compose ps`

Expected services:

- `db`
- `backend`
- `frontend`

Login Test

```
curl -X POST http://localhost:8000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username": "admin1", "password": "admin1pass"}'
```

Expected result:

- A JSON response containing `token`, `user_id`, `username`, and `role`.

Knowledge Base List Test

Use a token from login:

```
curl http://localhost:8000/api/knowledge-base \
  -H "Authorization: Bearer <token>"
```

Expected result:

- JSON array of knowledge base files.

Settings Ping Test

```
curl -X POST http://localhost:8000/api/settings/ping \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{"target": "chat"}'
```

Expected result:

- **ok: true** if the selected endpoint and model are reachable.

Client Handoff Checklist

Before client handoff, confirm:

- Docker stack starts successfully with **make up**.
- Frontend opens at **http://localhost:3000**.
- Backend is reachable at **http://localhost:8000**.
- Admin login works.
- User login works.
- Admin can upload a document.
- Uploaded document reaches **Indexed** status.
- User can compare a new document.
- Gap analysis dashboard renders.
- Chat returns answers with sources.
- Settings ping works for the selected model.
- Default credentials are replaced or client is instructed to replace them.
- **.env** is created and documented for the client.
- OpenAI API key is valid and billing/usage permissions are confirmed.

Future Enhancements and Product Roadmap Ideas

The application is already functional for the primary workflow. The same RAG architecture can be extended into adjacent document analysis domains, so the roadmap combines implementation hardening with domain expansion opportunities.

Platform Enhancements

- Persist runtime LLM settings to a secure database-backed configuration table.
- Add formal audit logs for upload, delete, comparison, and settings changes.
- Add a richer document preview experience for source excerpts.
- Add downloadable gap analysis exports from the frontend.
- Add more granular permissions if the client needs multiple knowledge bases or departments.
- Add production-grade CORS configuration.
- Add automated background retry and failure recovery for ingestion jobs.
- Add a formal admin user management screen.
- Expand Ollama validation with selected local models and client sample documents.

Domain Expansion Ideas

The same ingestion, retrieval, and comparison pattern can be adapted to new document categories by changing the extraction prompt, metadata schema, verdict definitions, and frontend report labels.

Expansion Area	Enhancement Idea
Contract review	Add clause extraction, clause-risk scoring, obligation mapping, and deviation reporting against standard templates
Policy and SOP impact	Compare new policies against existing SOPs and identify departments, procedures, or controls that require updates
Regulatory compliance	Map requirements against regulations such as HIPAA, HL7 FHIR, SOX, ISO, or internal compliance controls
Vendor/RFP evaluation	Compare vendor proposals against requirement baselines and generate objective compliance scores
Multi-domain prompt packs	Create selectable prompt packs for software requirements, contracts, SOPs, compliance, and procurement
Weighted scoring	Add severity, priority, risk, or business-value weighting to gap reports
Portfolio-level reporting	Summarize gaps across many uploaded documents, vendors, departments, or projects
Client-branded exports	Export final reports as PDF, Word, or PowerPoint using client templates

Conclusion

DocCompare is a strong client-ready foundation for requirements intelligence. It combines document ingestion, LLM-based extraction, semantic search, structured gap analysis, and source-backed chat in a practical workflow.

The application is especially useful because it works at the acceptance criteria level, which is the level at which business, QA, and delivery teams usually need to make decisions. The role-based design keeps the knowledge base controlled by admins, while still allowing regular users to run comparisons and ask questions.

With Docker deployment, OpenAI-compatible configuration, PostgreSQL/pgvector persistence, and a clear React frontend, the application is well-positioned for client demonstration, pilot usage, and future production hardening.